

Reusing SaccadeJEPA for Gaze Dynamics: From Joint-Embedding Predictive Architectures to Saliency-Driven Scanpaths

Emad Bahreinipour

Natural Interaction Course

University of Milan

May 2026

Abstract

SaccadeJEPA is an unpublished JEPA-style implementation in which a predictor learns to map one randomly sampled crop to the target-encoder representation of another crop. This report reverse-engineers that mechanism and asks whether it can be reused for explicit gaze-dynamics simulation with minimal architectural change. The adaptation replaces only the random cropper with **GazeCropper**, which generates saliency $\times$ inhibition-of-return fixation sequences while leaving the context encoder, EMA target encoder, predictor, and Huber + cycle + VICReg objective unchanged. On the FIND dataset, the patched model reduces validation prediction MSE from 0.30 to 0.046 after five CPU epochs ($\sim 85\%$ relative drop). This does not claim a complete learned human gaze policy; it shows that the original JEPA machinery remains trainable when random crop pairs are replaced by structured gaze transitions.

1 General Description of JEPA

1.1 The Basic Idea

A Joint-Embedding Predictive Architecture (JEPA) [3] is a self-supervised learning framework built around one central idea: instead of reconstructing missing data in pixel space, the model predicts missing information in a learned representation space. In plain terms, it does not try to reproduce every texture, colour fluctuation, or noise pattern in an image. It tries to predict the representation of what is missing.

This is different from pixel-reconstruction methods such as autoencoders, which must model all visible detail, including information that may be irrelevant for understanding the scene. It is also different from contrastive methods, which often depend on negative examples and hand-crafted augmentations. JEPA instead creates a predictive task directly from the input: encode one part, encode another part, and train a predictor to map the first representation to the second.

1.2 Core Principle: Prediction in Representation Space

Let $\mathbf{x}$ denote the observed, or context, part of an input, and let $\mathbf{y}$ denote the target part. The context encoder f_θ maps $\mathbf{x}$ to a latent representation, while the target encoder $f_{\bar{\theta}}$ maps $\mathbf{y}$ into the same representation space:

$$s_x = f_\theta(\mathbf{x}), \quad (1)$$

$$s_y = f_{\bar{\theta}}(\mathbf{y}). \quad (2)$$

The predictor receives the context representation and estimates the representation that the target encoder would assign to $\mathbf{y}$:

$$\hat{s}_y = g_\phi(s_x), \quad (3)$$

Training then minimises the distance between the predicted representation and the encoded target representation:

$$\mathcal{L} = d(\hat{s}_y, s_y), \quad (4)$$

where $d(\cdot, \cdot)$ is usually a squared-error or Huber-style distance. The important point is that the model never has to produce a pixel-level reconstruction $\hat{\mathbf{y}}$. The loss is computed only between representations, so the predictor is pushed to recover the target’s abstract state rather than its exact texture or noise.

1.3 The EMA Target Encoder

A practical issue in joint-embedding methods is representational collapse: both branches could learn nearly constant vectors, producing a low loss without useful visual features. JEPA reduces this risk by making the two branches asymmetric. The context encoder is trained by gradient descent, while the target encoder is updated only as a slow-moving copy of it.

Concretely, the target encoder parameters $\bar{\theta}$ are updated by an exponential moving average (EMA) of the context encoder parameters θ :

$$\bar{\theta} \leftarrow \tau \bar{\theta} + (1 - \tau) \theta, \quad (5)$$

with τ close to one. In plain terms, gradients do not directly modify the target branch. Instead, the target encoder follows the context encoder with a delay. This gives the predictor a slowly changing reference representation, while still allowing the target representation to improve over training.

1.4 I-JEPA as the Image Example

I-JEPA [1] is the canonical image-based instantiation of this principle. It divides an image into patch tokens, keeps large contiguous target blocks out of the context encoder’s input, and gives that encoder only the visible patches. The target encoder still computes representations for those target blocks, and the predictor learns to infer those target representations from the visible context.

What matters here is not every architectural detail of I-JEPA, but the pattern it establishes:

visible image region $\rightarrow$ predict representation of unseen image region.

Because I-JEPA predicts representations rather than pixels, it encourages the encoder to learn semantic and spatially meaningful features. This makes it a useful conceptual bridge to gaze: a fixation also exposes only part of the visual scene in high acuity, while the system must maintain a representation of what lies outside the current fixation.

1.5 Why JEPA Fits Gaze Modelling

The same formulation can be read as a gaze-transition problem. If $\mathbf{x}$ is a crop around the current fixation and $\mathbf{y}$ is a crop around the next fixation, the task becomes:

current fixation representation + movement information $\rightarrow$ predict
representation of next fixation.

This is why SaccadeJEPA is a useful starting point. Its original pretext task already predicts the representation of a shifted crop from another crop and an encoded movement. The adaptation in this report keeps that prediction problem but changes where the shifts come from: random affine perturbations are replaced by saliency-driven fixation transitions.

2 Adaptation for Attentive Vision and Gaze

2.1 What Original SaccadeJEPA Provides

The starting point is the `SaccadeJepa` model from the `LumenPallidum JEPA` repository [4]. It is an unpublished JEPA-style variant, separate from the repository’s standard `ViTJepa` implementation. Its pretext task is simple: given two spatially related crops from the same image, predict the target encoder’s representation of one crop from the context encoder’s representation of the other.

In plain words, original SaccadeJEPA does this:

image $\rightarrow$ random affine view + original center crop
 $\rightarrow$ encode $\rightarrow$ predict target representation.

The main components are already useful for gaze modelling:

- **Context encoder:** in the default configuration, a `ConvNeXt-Tiny` network that encodes the first crop.
- **Target encoder:** an EMA copy of the context encoder that encodes the second crop without gradient updates.
- **Movement encoding:** the cropper returns a NeRF-style sinusoidal encoding of the sampled rotation and translation, which is then projected into the representation space before prediction.
- **Predictor:** a three-layer MLP that receives the context representation plus the movement embedding and predicts the target representation.

The original cropper is the weak part for gaze. **SaccadeCropper** samples one random affine transformation per image. The sampled movement is not conditioned on image content, does not form a sequence, and does not remember where the model has already looked. It is useful as a representation-learning pretext task, but it is not yet an explicit gaze simulator.

2.2 What Needs to Change for Gaze

For the explicit gaze simulation targeted in this report, the model needs three behaviours that original SaccadeJEPA does not provide:

- **Content-driven target selection.** The next crop should depend on the image, not on a uniform random draw.
- **Sequential fixations.** The model should produce a short scanpath, not only one independent crop pair.
- **Inhibition of return.** Recently visited locations should be temporarily suppressed so the model does not repeatedly sample the same point [2].

The conservative design choice is to change only the component responsible for choosing and producing crops. The encoder, target encoder, EMA update, predictor, affine embedder, and loss are kept from SaccadeJEPA. This keeps the adaptation easy to interpret: if the patched model learns, the learning signal still comes from the original JEPA machinery, but it is now applied to saliency-guided fixation transitions rather than random affine perturbations.

2.3 The Cropper Substitution

The adaptation is implemented in the project’s JEPA-reuse module. The `make_gaze_jepa` factory first builds a normal **SaccadeJepa** instance and then replaces one attribute:

```
sj.saccade_cropper = GazeCropper(...)
```

This is the central architectural change of the project. Before the swap, the model receives one randomly transformed crop paired with the original centre crop. After the swap, **GazeCropper** internally generates a short saliency-driven fixation sequence, converts it into consecutive fixation transitions, and exposes those transitions through the same crop-pair interface:

```
image -> saliency map -> fixations -> transition crop pairs
                                -> JEPA prediction.
```

GazeCropper preserves the same return signature as the upstream **SaccadeCropper**:

$$(\mathbf{view}_1, \mathbf{view}_2, \mathbf{affines}) = \text{cropper}(\mathbf{x}). \quad (6)$$

Because the rest of **SaccadeJepa.forward** sees the same three tensors, the upstream forward pass does not need to be rewritten.

2.4 How GazeCropper Works

For each image, **GazeCropper** performs a short explicit gaze loop:

1. Compute a saliency map from the image.
2. Initialise an inhibition-of-return mask to ones.
3. For T steps, sample the next fixation from saliency $\times$ IOR.
4. After each fixation, suppress that location with a Gaussian inhibition well and let older suppressions decay.
5. Convert the T fixations into $T - 1$ transitions ($\text{fix}_t \rightarrow \text{fix}_{t+1}$).

The saliency map is computed once for the input frame and reused across the whole T -fixation walk; between steps, only the inhibition-of-return mask changes. This is a static-frame approximation of gaze dynamics, not a temporal video model. Figure 1 shows this loop on a FIND frame: the inhibition mask deepens at each visited location, and the saliency $\times$ IOR map is the distribution from which each fixation is drawn.

In the experiments below, $T = 5$, so each image produces four transitions. These transitions are flattened into the batch dimension, giving $B(T - 1)$ crop pairs to the original JEPA encoder and predictor. Each transition contributes a 128×128 crop at the current fixation, a 128×128 crop at the next fixation, and the normalised displacement between them. The predictor is therefore trained on:

`current fixation crop + movement embedding  $\rightarrow$  next fixation representation.`

The saliency source is pluggable: any module that maps an RGB image batch to a normalised saliency map can be used. This includes random saliency, centre bias, classical spectral-residual saliency, or Arash Khosropour’s trained ResNet saliency model. The ResNet model is the default for the final experiments; it is trained separately on FIND fixation maps and then frozen during JEPA training.

2.5 Compatibility with SaccadeJEPA

The substitution is safe because **GazeCropper** respects the two contracts expected by the upstream model.

First, the embedding dimension is unchanged. The upstream affine embedder expects a 36-dimensional NeRF-style movement embedding. This comes from translation order $L_t = 4$ and rotation order $L_r = 10$:

$$\text{affine_embed_dim} = L_t \cdot 4 + L_r \cdot 2 = 36. \quad (7)$$

GazeCropper reads these orders from the original cropper and produces the same embedding width. Rotation is set to zero, because gaze shifts translate the fixation window but do not rotate the world.

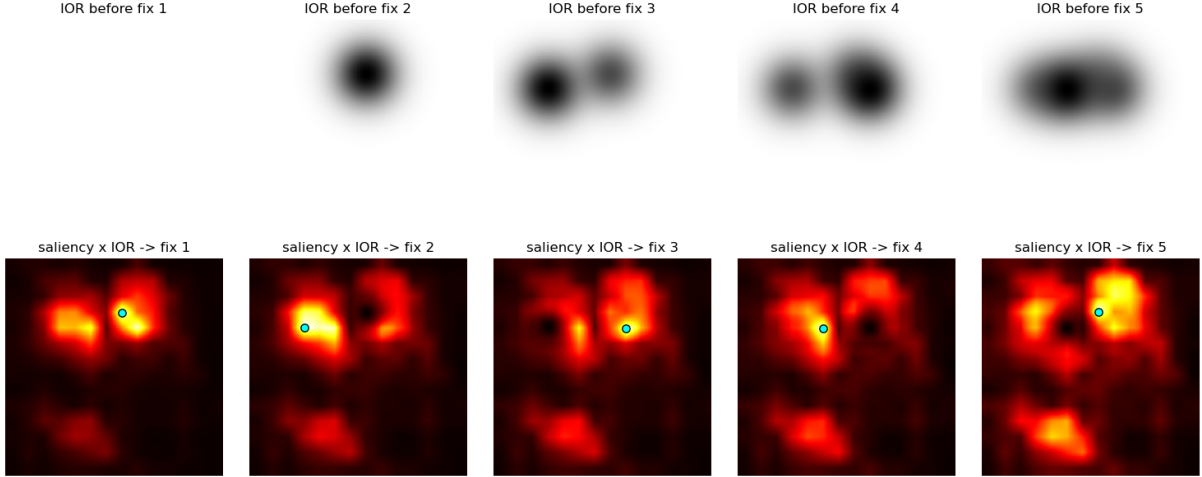


Figure 1: The **GazeCropper** loop on a single FIND frame, $T = 5$ fixations. **Top:** the inhibition-of-return mask before each fixation; previously visited locations are suppressed (dark) and older suppressions decay back toward one. **Bottom:** the saliency $\times$ IOR map at each step, which is the distribution the next fixation is sampled from. Because IOR pushes the sampler away from recently visited regions, the fixations spread across the salient structure of the frame instead of collapsing onto a single peak.

Second, the encoder inputs remain symmetric. Both the context encoder and the target encoder still receive 3-channel 128×128 RGB crops. This matters because the target encoder is an EMA copy of the context encoder. An earlier integrated-controller design used a 6-channel fovea-plus-periphery context branch with a 3-channel target branch; that would have made the EMA path asymmetric. The final design avoids that issue by keeping the upstream 3-channel crop interface.

2.6 Losses Reused from SaccadeJEPA

The training objective is also reused from SaccadeJEPA. The main term is a Huber prediction loss between the predictor output and the EMA target encoder’s representation of the target crop. The upstream forward pass may randomly swap the two views, so the model is trained to predict both directions of the fixation transition.

The cycle-consistency term is kept as well: after predicting one crop’s representation from the other, the inverse movement embedding is used to ask the predictor to approximately recover the original context representation. Finally, the VICReg-style variance and covariance regularisers are kept on the context representations to discourage collapse.

Thus the adaptation is deliberately minimal. The source of the two views changes from a random affine perturbation to a saliency-driven gaze transition; the encoder, target encoder, predictor, EMA path, and loss recipe remain the same.

3 Gaze-Transition Simulation and Results

The simulation in this section is a self-supervised gaze-transition prediction task on FIND. The model receives a static video frame, uses **GazeCropper** to generate a short saliency-driven fixation sequence, and trains SaccadeJEPA’s predictor to anticipate the representation of the target fixation crop. This does not yet claim to reproduce human scanpaths

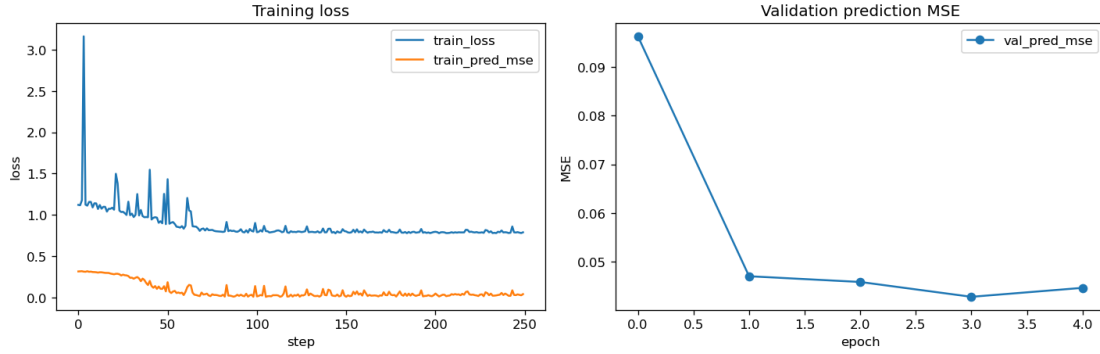


Figure 2: Training loss and prediction MSE for the patched **SaccadeJepa** on FIND. Five epochs, batch size 4, four frames per training video, on CPU. The total loss combines a Huber prediction term, a cycle term, and the VICReg variance/covariance regulariser. The prediction MSE is read directly from the predictor against the EMA target encoder. Per-batch values are shown lightly; the heavy line is a running mean.

exactly. It tests a more basic question: after the cropper substitution, can the original JEPA machinery still learn?

3.1 Simulation Setup

Frames are decoded from FIND raw MP4 videos and resized to 200×200 . The shared split is 50 training videos, 6 validation videos, and 6 test videos. For the demonstration run, each training video contributes four sampled frames per epoch. With batch size 4 and $T = 5$ fixations per frame, the predictor sees 16 fixation transitions per batch.

Training uses the same loss family described in Sec. 2.6: Huber prediction loss, cycle consistency, and VICReg variance/covariance regularisation. AdamW is used with learning rate $5 \cdot 10^{-5}$ and EMA decay $\tau = 0.996$. The only practical adjustment is that the VICReg weights are reduced for the small CPU batch size: $\lambda_{\text{var}} = 1.0$, $\lambda_{\text{cov}} = 0.1$, and $\gamma = 1.0$. The upstream defaults were tuned for a much larger effective batch.

The run lasts five CPU epochs. Figure 2 shows that both total loss and prediction MSE decrease over training. The key curve is prediction MSE, because it measures how close the predictor is to the EMA target representation of the target fixation crop.

3.2 Quantitative Results

The main result is the validation prediction MSE (Table 1). Before training, the patched model gives a validation MSE of 0.2996. After five epochs, the matched validation pass gives 0.0462. This is an 84.6% reduction. The result means that the predictor learned to anticipate the target fixation representation under the saliency-driven cropper.

3.3 Qualitative Scanpath Simulation

Figure 3 shows four scanpaths sampled on held-out FIND test frames. The cropper itself is not trained by the JEPA loss; its fixations come from the frozen saliency source and the IOR mask. The figure is therefore not evidence that the model has learned a human scanpath policy. Its role is more limited: it shows that the substituted cropper

Phase	Prediction MSE
Untrained, validation pool ($n_{\text{val}}=24$)	0.2996
Validation, end of epoch 1	0.0963
Validation, end of epoch 2	0.0471
Validation, end of epoch 3	0.0459
Validation, end of epoch 4	0.0428
Validation, end of epoch 5	0.0447
Trained, validation pool (matched val pass on <code>final.pt</code>)	0.0462

Table 1: Predictor mean-squared error against the EMA target encoder, before and after a five-epoch training run on FIND. All numbers are averaged across 24 validation frames from the six held-out validation videos, with each frame yielding four fixation transitions ($T=5$). The saliency source is Arash’s trained **ResNetSaliency**. The last row re-loads the final checkpoint and replays the validation pass in matched conditions. The training-time number recorded at end of epoch 5 (0.0447) and the post-hoc reproduction (0.0462) agree closely despite stochastic fixation sampling.



Figure 3: Fixation sequences produced by **GazeCropper** on four FIND test frames. Numbered red markers are the five fixations in temporal order; lines connect consecutive fixations. The saliency source is Arash’s trained **ResNetSaliency** (ResNet-18 backbone with a 1×1 -convolution head, drawn from the shared `gazejepa.saliency` package), and the inhibition mask uses the same $\sigma_{\text{IOR}}=20$ px and decay 0.7 as in the training run.

produces structured, sequential fixations on real video frames, and that the crops passed into SaccadeJEPA are visually meaningful.

3.4 Interpretation

These results answer the feasibility question positively. With a one-component substitution, SaccadeJEPA can be made to operate on explicit gaze transitions, and the original prediction machinery still converges. The decrease in MSE is the same kind of JEPA signal as in the original model: the predictor approaches the EMA target representation. The difference is that the two views now come from consecutive saliency-driven fixations rather than one random affine crop.

The result should be interpreted carefully. The model is not trained with a human scanpath loss, and it does not receive gradients through the discrete fixation choices. It is therefore better described as a reusable JEPA backbone for gaze-transition prediction, not as a complete learned gaze policy.

3.5 Limitations

The main limitations are:

- **Static frames.** FIND is a video dataset, but this simulation runs $T = 5$ fixations on a single frozen frame rather than modelling gaze over consecutive video frames.
- **No fixation duration.** The model predicts where the next crop is, not how long the eye stays there.
- **Frozen saliency.** Arash’s ResNetSaliency is used as a fixed prior. It is not updated by the JEPA loss.
- **Fixed gaze-loop parameters.** The number of fixations ($T = 5$), IOR width ($\sigma_{\text{IOR}} = 20$ px), and IOR decay (0.7) are fixed hyperparameters rather than learned from data.
- **Small training budget.** The run is five epochs on CPU, so it is a feasibility check rather than a tuned final model.

4 Conclusion

This report asked whether SaccadeJEPA can be reused in a less obscure way for explicit gaze simulation. The answer is yes, within a carefully limited scope. Original SaccadeJEPA already contains the useful JEPA machinery: a context encoder, an EMA target encoder, a movement embedding, a predictor, and a representation-level prediction loss. Its limitation is the source of the views: one random crop pair per image.

The proposed adaptation changes that source of views. Replacing **SaccadeCropper** with **GazeCropper** turns the model from:

image -> random crop pair -> predict representation

into:

image -> saliency-driven fixation sequence -> predict next fixation.

The experiment on FIND shows that this substitution is compatible with the upstream loss: validation prediction MSE drops from 0.2996 to 0.0462 after five CPU epochs. This is not a claim that the model fully matches human gaze, but it is strong feasibility evidence that SaccadeJEPA can be reused as a representation-space predictor over explicit gaze transitions.

References

- [1] M. Assran, Q. Duval, I. Misra, P. Bojanowski, P. Vincent, M. Rabbat, Y. LeCun, and N. Ballas. Self-supervised learning from images with a joint-embedding predictive architecture. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023.
- [2] R. M. Klein. Inhibition of return. *Trends in Cognitive Sciences*, 4(4):138–147, 2000.

- [3] Y. LeCun. A path towards autonomous machine intelligence. *OpenReview*, 2022. <https://openreview.net/pdf?id=BZ5a1r-kVsf>.
- [4] LumenPallidium. JEPA: a PyTorch implementation of joint-embedding predictive architectures, including SaccadeJEPA. Software repository, 2024. <https://github.com/LumenPallidium/jepa>, accessed 2026.